



UNIVERSITÀ
DEGLI STUDI
DI PALERMO



Introduzione a Javascript

Corso di Programmazione Web e Mobile
a.a. 2025/2026

Ing. Luca Cruciata

Sommario

- Javascript: linguaggio di scripting a oggetti
 - Breve storia
 - Filosofia di programmazione
 - Come si inserisce uno script nella pagina web
- Variabili, tipi semplici e operatori
- Le stringhe, i numeri e la gestione delle funzioni matematiche
- Strutture di controllo del flusso del programma
- Funzioni

JavaScript: caratteristiche

- Linguaggio di **scripting** orientato agli *oggetti* e agli *eventi*
 - Utilizzato nella programmazione Web lato client per la creazione di effetti dinamici interattivi tramite funzioni di script invocate da *eventi* innescati a loro volta in vari modi dall'utente (mouse, tastiera, caricamento della pagina ecc...)
 - Descrive l'ambiente del client web tramite un apposito insieme di *oggetti* che possono essere manipolati dallo script
- Linguaggio interpretato
- Sintassi simile a Java, C, C++
- Debolmente tipizzato
 - I tipi delle variabili sono inferiti, in fase di assegnamento, dalla valutazione delle espressioni

JavaScript, cosa fa?

Javascript può:

- Cambiare il contenuto HTML
- Cambiare gli attributi HTML
- Cambiare gli stili HTML (CSS)
- Nascondere/mostrare elementi HTML
- Aggiungere/rimuovere elementi HTML
- Reagire a eventi
- Comunicare con i server

Inserimento di uno script nella pagina web

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    ...
```

```
    <script src="myscript.js"></script>
```

```
    <script>
```

codice Javascript

```
  </script>
```

```
  <style></style>
```

```
</head>
```

```
<body>
```

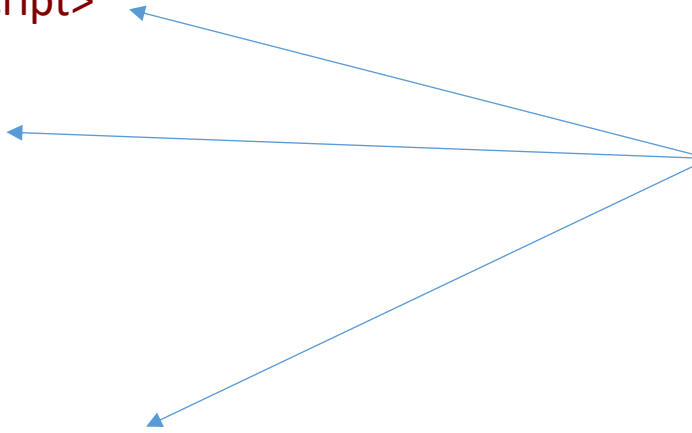
codice HTML

```
<script src="myscript1.js"></script>
```

```
</body>
```

```
</html>
```

Possibili alternative, per
inserimento del codice js



JavaScript, visualizzazione

È possibile scrivere all'interno di:

- Un elemento HTML, usando `innerHTML`.
- Una alert box, usando `window.alert()`
- Il documento HTML, usando `document.write()`
 - Usato principalmente durante il caricamento iniziale; se usato successivamente, può cancellare l'intero contenuto.
- La log console del browser, usando `console.log()`

JavaScript, sintassi

- JavaScript è un linguaggio case sensitive
- Il punto e virgola a fine frase è opzionale, ma consigliato. Il browser dovrebbe interpretare l'andata a capo come la fine della linea di codice
- Le linee di codice possono essere raggruppate insieme in blocchi, utilizzando la parentesi graffa sinistra { all'inizio del blocco di codice e quella destra } alla fine.

Tipi di dati

L'operatore `typeof` ritorna il tipo del dato di una espressione

<code>typeof "John"</code>	<code>// Returns string</code>
<code>typeof 3.14</code>	<code>// Returns number</code>
<code>typeof NaN</code>	<code>// Returns number</code>
<code>typeof false</code>	<code>// Returns boolean</code>
<code>typeof [1, 2, 3, 4]</code>	<code>// Returns object</code>
<code>typeof {name:'John', age:34}</code>	<code>// Returns object</code>
<code>typeof new Date()</code>	<code>// Returns object</code>
<code>typeof function () {}</code>	<code>// Returns function</code>
<code>typeof myCar</code>	<code>// Returns undefined (if myCar is not declared)</code>
<code>typeof null</code>	<code>// Returns object</code>

Tipi di dati

- Per questi tipi si definiscono dei letterali e non è necessario invocare il costruttore

// Number

23

-32.567e-10

Infinity // è un valore numerico

Nan // risultato di una forma

// indeterminata ad es. 0/0

- Letterali dei tipi semplici

// String

'pippo'

"pluto"

// Boolean

true

false

Tipi di dati

- Altri letterali delle stringhe

\'

\"

\\

Code	Result
\b	Backspace
\f	Form Feed
\n	New Line
\r	Carriage Return
\t	Horizontal Tabulator
\v	Vertical Tabulator

Gestione stringhe

```
console.log("questo è uno \\")  
console.log("questo è uno \\'")  
console.log("questo è un \\\")
```

```
'questo è uno \'  
"questo è uno '  
'questo è un '"
```

```
console.log("questo è uno \")
```

```
SyntaxError: Unterminated string
```

Il carattere \ serve come carattere di escape per introdurre le **sequenze di escape** (\n, \t, ...)

Operatori + sulla stringhe

- L'operatore + può essere usato per concatenare stringhe o valori testuali.
- Esempio:
 - `txt1 = "What a very";`
 - `txt2 = "nice day";`
 - `txt3 = txt1 + txt2;`
- Risultato:
 - `"What a verynice day".`

Tipi di dati

- Letterali particolari
 - `null`, valore dell'oggetto nullo
 - `undefined`, identificatore dichiarato e non inizializzato

JavaScript, variabili

- Le variabili (in JavaScript) vengono utilizzate per memorizzare dei valori
- *Una variabile è un «contenitore di dati»*
- Una variabile può avere un nome simbolico, come *x*, o un nome più descrittivo, come *carname*
- Regole di JavaScript per i nomi delle variabili:
 - I nomi delle variabili sono Case Sensitive
 - I nomi delle variabili devono cominciare con una lettera o con i simboli \$ o _
- Le variabili possono contenere valori di qualsiasi **tipo**

Variabili

- Ci si riferisce alla creazione di variabili con l'espressione *«dichiarazione di una variabile»*
- Una variabile viene dichiarata con la parola chiave **var**
 - **var** x;
 - **var** carname;
- Dopo la dichiarazione fatta sopra, le variabile sono vuote
- È possibile assegnare dei valori alle variabili quando vengono dichiarate:
 - **var** x=5;
 - **var** carname="Volvo";

Variabili

- Una variabile si dichiara con
 - **var**, con ambito di visibilità globale
 - **let**, con ambito di visibilità locale rispetto al blocco di definizione

```
var foo, bar = 12;
```

```
console.log('bar: ' + bar);  
console.log('foo: ' + foo);
```

```
if (!foo) {  
    let baz = bar;  
    console.log('baz: ' + baz);  
    //let baz = 2;  
}
```

```
var bar = "pippo";  
console.log('bar: ' + bar);  
console.log('baz: ' + baz);
```

Variabili

```
var foo, bar = 12;
```

```
console.log('bar: ' + bar);  
console.log('foo: ' + foo);
```

```
if (!foo) {  
    let baz = bar;  
    console.log('baz: ' + baz);  
}
```

```
var bar = "pippo";  
console.log('bar: ' + bar);  
console.log('baz: ' + baz);
```

Quale sarà l'output del seguente codice?

```
bar: 12  
foo: undefined  
baz: 12  
bar: pippo  
✖ ▶ Uncaught ReferenceError: baz is not defined  
   at <anonymous>:12:23
```

Variabili

La definizione tramite *var* permette:

```
var a = 1;  
var a = 2;
```

Mentre, con *let* **non permette** la definizione della stessa variabile:

```
let b = 1;  
let b = 2; ERRORE
```

Variabili

La definizione tramite *var* definisce l'oggetto:

```
console.log(c); undefined  
var c = 5;
```

Mentre, con *let* otteniamo:

```
console.log(d); errore  
let d = 5;
```

Variabili

Quando lavoriamo nella console del browser:

```
var x = 1;  
let y = 2;
```

```
console.log(window.x); 1  
console.log(window.y); undefined
```

Costanti

Una variabile si dichiara con **const**, una volta inizializzata non può più essere modificata.

```
const myNumber = 123456;
```

```
myNumber = 6      ERRORE!
```

Operatori - aritmetici

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Operatori - assegnamento

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operatori - confronto

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Operatori - logici

Operator	Description
&&	logical and
	logical or
!	logical not

Operatori – bit a bit

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
	OR	5 1	0101 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	Zero fill left shift	5 << 1	0101 << 1	1010	10
>>	Signed right shift	5 >> 1	0101 >> 1	0010	2
>>>	Zero fill right shift	5 >>> 1	0101 >>> 1	0010	2

Operatori – identificazione del tipo

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

Per una lista completa con precedenza e associatività, si consulti il portale [Javascript MDN](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators)

Instanceof esempio

```
function Car(make, model, year) {  
  this.make = make;  
  this.model = model;  
  this.year = year;}  
  
const auto = new Car("Honda", "Accord", 1998);  
  
console.log(auto instanceof Car);  
console.log(auto instanceof Object);
```

Quale sarà l'output del seguente codice?

True
True

Conversioni automatiche di tipo

- Javascript è caratterizzato dal fatto che determina i tipi al momento del calcolo delle espressioni.
- La *type coercion* è l'insieme dei meccanismi automatici per effettuare la conversione
 - I risultati non sono sempre immediatamente predicibili
 - Quanto fa "5" + 1 e "5" * 1 ?

Conversioni automatiche di tipo

<code>var a= "5"</code>	
<code>console.log(a+1)</code>	<code>'51'</code>
<code>console.log(a*1)</code>	<code>5</code>
<code>console.log(a-3)</code>	<code>2</code>
<code>console.log(a/2)</code>	<code>2.5</code>
<code>var b=1</code>	
<code>console.log(b+"1")</code>	<code>'11'</code>
<code>console.log(b*"2")</code>	<code>2</code>
<code>console.log(b-"3")</code>	<code>-2</code>
<code>console.log(a/"2".5")</code>	<code>2</code>

Conversioni automatiche di tipo

- Per quanto riguarda gli operatori booleani in cascata per la valutazione di più espressioni essi usano una *valutazione cortocircuito*
- Gli operandi sono valutati da sinistra a destra *andando avanti solo fino a quando non vi è certezza del risultato*
- La parte restante dell'espressione non è valutata

Conversioni automatiche di tipo

OPERATORE AND (&&)

- `console.log(false && alert("Non viene eseguito!"));` → **false**
- `console.log(true && "Hello" && 42);` → **42**

Se il primo operando è false, il resto non viene valutato.

OPERATORE OR (||)

- `console.log(true || alert("Non viene eseguito!"));` → **true**
- `console.log(false || 0 || "Ciao");` → «Ciao»

Se il primo operando è true, il resto non viene valutato.

JavaScript Arrays

- Un array è una variabile speciale che può contenere più di un valore.
 - `Const cars = ["Saab", "Volvo", "BMW"];`
- In generale:
 - `const array_name = [item1, item2, ...];`
- Accesso ad un elemento dell'array effettuato attraverso le parentesi quadre `[]` ed il numero di indice:
 - `Const cars = ["Saab", "Volvo", "BMW"];`
 - `let car = cars[0]`
- Sovrascrittura di un elemento:
 - `Const cars = ["Saab", "Volvo", "BMW"];`
 - `cars[0] = "Opel";`

Javascript Arrays

```
let numeri2 = new Array(1, 2, 3, 4, 5);
```

```
let numeri = [1, 2, 3, 4, 5];
```

Metodi di accesso classici:

```
numeri.push(4); // Aggiunge alla fine → [ 1, 2, 3, 4, 5, 4 ]
```

```
numeri.unshift(0); // Aggiunge all'inizio → [ 0, 1, 2, 3, 4, 5, 4 ]
```

```
numeri.pop(0); // Ritorna l'ultimo elemento e lo toglie dall'array
```

Metodi Javascript Arrays

map() serve per modificare ogni elemento e ottenere un nuovo array

```
const numeri = [1, 2, 3];
```

```
const raddoppiati = numeri.map(n => n * 2);
```

```
console.log(raddoppiati); // [2, 4, 6]
```

Metodi Javascript Arrays

filter() serve per mantenere solo gli elementi che rispettano una condizione data

```
const numeri = [1, 2, 3, 4];
```

```
const pari = numeri.filter(n => n % 2 === 0);
```

```
console.log(pari); // [2, 4]
```

Metodi Javascript Arrays

forEach() esegue codice

```
const numeri = [1, 2, 3];
```

```
numeri.forEach(n => {  
  console.log(n * 2);  
});
```

Metodi Javascript Arrays

reduce() esegue codice

```
const numeri = [1, 2, 3, 4];
```

```
const somma = numeri.reduce((acc, n) => acc + n, 0);
```

```
console.log(somma); // 10
```

Metodi Javascript Arrays

Gestione dei for nei casi in cui dobbiamo aspettare il risultato di un'operazione:

- `numeri.forEach(async (n) => {
 await fetch(...); //non aspetta davvero
});`
- `for (let n of numeri) {
 await fetch(...); //aspetta
}`

Combinazione di metodi

```
const utenti = [  
  { nome: "Luca", eta: 17 },  
  { nome: "Anna", eta: 25 },  
  { nome: "Marco", eta: 30 }  
];  
  
// prendi solo maggiorenni e i loro nomi  
const nomi = utenti.filter(u => u.eta >= 18).map(u => u.nome);  
  
console.log(nomi); // ["Anna", "Marco"]
```

Per riassumere:

- vuoi modificare valori → **map**
- vuoi selezionare elementi → **filter**
- vuoi calcolare qualcosa → **reduce**

Oggetti

- Dichiarazione

```
var nomeOggetto = {  
  proprietà: valore,  
  proprietà: valore,  
  metodo: function(parametro, parametro, ...) { codice },  
  ...  
}
```

Oggetti in JavaScript

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  id      : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

- La parole chiave **this** si riferisce all'oggetto stesso. In questo caso, rappresenta il firstName di questo oggetto person

Oggetti

- Gli oggetti definiti dall'utente sono di classe Object
- Stringhe, array, date, espressioni regolari, numeri, booleani sono oggetti predefiniti in Javascript
- Accesso ad un elemento di un oggetto

nomeOggetto["proprietà"]

nomeOggetto.proprietà

nomeOggetto.metodo(lista parametri)

Tipi di dati

- Tutti i tipi di dati in Javascript sono oggetti, derivati dalla classe base `Object`
 - `Number`
 - `String`
 - `Boolean`
 - `Date`
 - `Array`
 - `Function`
 - `Math`

Number

Number.toLocaleString

Number.valueOf

Number.apply

Number.call

Number.constructor

Number.toString

Number.EPSILON

Number.MIN_SAFE_INTEGER

Number.NEGATIVE_INFINITY

Number.NaN

Number.isInteger

Number.isSafeInteger

Number.length

Number.parseInt

Number.arguments

Number.caller

Number.MAX_SAFE_INTEGER

Number.MIN_VALUE

Number.POSITIVE_INFINITY

Number.isNaN

Number.name

Number.prototype

Number.bind

Number.MAX_VALUE

Number.isFinite

Number.parseFloat

Number

```
x=new Number(8)
[Number: 8]
> x.
```

```
x.constructor
x.toFixed
x.toPrecision
x.valueOf
```

```
x.toExponential
x.toLocaleString
x.toString
```

String

```
s=new String('pippo')  
[String: 'pippo']  
> s.
```

s.toLocaleString

s.anchor

s.charAt

s.constructor

s.fontsize

s.lastIndexOf

s.matchAll

s.repeat

s.small

s.sub

s.toLocaleLowerCase

s.toUpperCase

s.trimRight

s.big

s.charCodeAt

s.endsWith

s.includes

s.link

s.normalize

s.replace

s.split

s.substr

s.toLocaleUpperCase

s.trim

s.trimStart

s.blink

s.codePointAt

s.fixed

s.indexOf

s.localeCompare

s.padEnd

s.search

s.startsWith

s.substring

s.toLowerCase

s.trimEnd

s.valueOf

s.bold

s.concat

s.fontcolor

s.italics

s.match

s.padStart

s.slice

s.strike

s.sup

s.toString

s.trimLeft

Math

Math.

Math.E
Math.LOG2E
Math.abs
Math.asinh
Math.cbrt
Math.cosh
Math.fround
Math.log10
Math.min
Math.sign
Math.tan

Math.LN10
Math.PI
Math.acos
Math.atan
Math.ceil
Math.exp
Math.hypot
Math.log1p
Math.pow
Math.sin
Math.tanh

Math.LN2
Math.SQRT1_2
Math.acosh
Math.atan2
Math.clz32
Math.expm1
Math.imul
Math.log2
Math.random
Math.sinh
Math.trunc

Math.LOG10E
Math.SQRT2
Math.asin
Math.atanh
Math.cos
Math.floor
Math.log
Math.max
Math.round
Math.sqrt

Date

// Vecchio modo (Date)

```
const vecchiaData = new Date(2026, 2, 15);  
console.log(vecchiaData.toDateString()); // Sun Mar 15 2026
```

// Nuovo modo (Temporal)

```
const nuovaData = Temporal.PlainDate.from({ year: 2026, month: 3,  
day: 15 });  
console.log(nuovaData.toString()); // 2026-03-15
```

Date

Vecchio modo (Date) - Molto pronò a errori

```
const d = new Date(2026, 2, 15);  
d.setMonth(d.getMonth() + 3);  
d.setDate(d.getDate() + 14);
```

Nuovo modo (Temporal) - Leggibile e immutabile

```
const dataIniziale = Temporal.PlainDate.from('2026-03-15');  
const dataFinale = dataIniziale.add({ months: 3, weeks: 2 });  
console.log(dataFinale.toString()); // 2026-06-29
```

Date, Temporal

Creare un momento preciso a Roma

```
const roma = Temporal.Now.zonedDateTimeISO('Europe/Rome');
```

Convertirlo istantaneamente a Tokyo

```
const tokyo = roma.withTimeZone('Asia/Tokyo');  
console.log(`A Roma sono le: ${roma.toString()}`);  
console.log(`A Tokyo sono le: ${tokyo.toString()}`);
```

Date, Temporal

Temporal.PlainDate: Solo anno, mese, giorno (es. un compleanno)

Temporal.PlainTime: Solo ora, minuti, secondi (es. la sveglia delle 7:00)

Temporal.PlainDateTime: Data e ora senza fuso orario (es. un appuntamento in calendario senza specificare la zona)

Temporal.ZonedDateTime: Tutto quello che serve per eventi globali (data, ora, fuso orario, offset)

Temporal.Duration: Un lasso di tempo (es. "durata della corsa: 45 minuti")

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal

Strutture di controllo del flusso del programma

- Scelta tra due alternative

```
if ( condizione ) {  
    codice da eseguire se la condizione è vera  
} else {  
    codice da eseguire se la condizione è falsa  
}
```

Strutture di controllo del flusso del programma

- Scelta tra due alternative (continua)

```
if ( condizione ) {  
    codice da eseguire se la condizione è vera  
}
```

Strutture di controllo del flusso del programma

- Scelta tra più alternative

```
if ( 1^ condizione ) {  
    codice da eseguire se la 1^ condizione è vera  
} else if ( 2^ condizione ) {  
    codice da eseguire se la 2^ condizione è vera mentre la prima  
    è falsa  
} else if ...  
...  
} else {  
    codice da eseguire se tutte le condizioni precedenti  
    sono false  
}
```

Strutture di controllo del flusso del programma

- Scelta tra più alternative

```
switch( espressione da valutare una volta e che ritorna un valore ) {  
    case valore1:  
        codice da eseguire se l'espressione assume valore pari a valore1  
        break;  
    case valore2:  
        codice da eseguire se l'espressione assume valore pari a valore2  
        break;  
    ...  
    default:  
        codice da eseguire se l'espressione non assume nessuno dei valori previsti  
}
```

Strutture di controllo del flusso del programma

- Iterazioni

```
for ( inizializzazione; termine; aggiornamento) {  
    codice da eseguire  
}
```

Iterazioni

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>

<p id="demo"></p>

<script>
const cars = ["BMW", "Volvo", "Saab", "Ford", "Fiat",
"Audi"];

let text = "";
for (let i = 0; i < cars.length; i++) {
  text += cars[i] + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript For Loop

BMW
Volvo
Saab
Ford
Fiat
Audi

For loop in

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For In Loop</h2>

<p id="demo"></p>

<script>
const person = {fname:"John", lname:"Doe", age:25};

let txt = "";
for (let x in person) {
    txt += x + ": " + person[x] + ", ";
}

document.getElementById("demo").innerHTML = txt;
</script>

</body>
</html>
```

JavaScript For In Loop

fname: John, lname: Doe, age: 25,

Strutture di controllo del flusso del programma

- Iterazioni

```
while( condizione di iterazione ) {  
    codice da eseguire se la condizione è vera  
}
```

Strutture di controllo del flusso del programma

- Iterazioni

```
do {  
    codice da eseguire  
} while ( condizione di iterazione )
```

Strutture di controllo del flusso del programma

- `break` forza l'uscita dall'iterazione
- `continue` forza l'iterazione successiva

Strutture di controllo del flusso del programma

- Oggetti iterabili
 - Sono quegli oggetti che si prestano strutturalmente ad essere scanditi elemento per elemento:
 - `Array`, `String`, `NodeList` (classe del DOM), oggetti definiti dall'utente ...

`for` (*chiave* in *iterabile*) – itera sugli indici/chiavi/proprietà

Strutture di controllo del flusso del programma

- Oggetti iterabili
 - Sono quegli oggetti che si prestano strutturalmente ad essere scanditi elemento per elemento:
 - `Array`, `String`, `NodeList` (classe del DOM), oggetti definiti dall'utente ...

for (*elemento* of *iterabile*)

– itera sui valori di un oggetto iterabile con struttura assimilabile agli

`Array` (*non* gli oggetti definiti dall'utente) generando così un *iteratore*

Strutture di controllo del flusso del programma

- *Iteratore*

- Struttura dati che consente di accedere agli elementi di una sequenza attraverso un'unica operazione di accesso all'elemento, senza possibilità di reinizializzare la sequenza

Funzioni

- Dichiarazione

```
function nomeFunzione(parametro, parametro, ...) {  
    codice da eseguire  
    return risultato  
}
```

- Se non c'è `return` la funzione non ritorna nulla, cioè il risultato è `void`

Funzioni

- Una funzione è un oggetto di tipo `Function`, quindi è una variabile che può essere dichiarata anche

```
var nomeFunzione = function (parametro, ...) {  
    codice da eseguire  
    return risultato  
}
```

Funzioni

- Dichiarazione di *funzione arrow*

```
var nomeFunzione = (parametro, ...) => {  
    codice da eseguire  
    return risultato  
}
```

- Buona notazione per piccole funzioni

Funzioni

- Il passaggio dei parametri è per valore, quindi con copia locale dell'oggetto passato come parametro

```
function f(x) {  
    x = x.replace(/p/g, '*');  
    console.log(x);  
}
```

```
> var y = 'Pippo'  
'Pippo'
```

```
> f(y)  
'Pi**o'
```

```
> y  
'Pippo'
```

Funzioni

- Closure:
 - La funzione fa da wrapper ad un'altra funzione, restituita come risultato, che conserva al suo interno eventuali variabili locali che si vogliano conservare

```
function creaContatore() {  
  let contatore = 0; // Variabile privata  
  
  return function() {  
    contatore++;  
    console.log(contatore);  
  };  
}
```

```
const conta = creaContatore();  
conta(); // 1  
conta(); // 2  
conta(); // 3
```

Esempio Closure

- `function delayMessage(messaggio, tempo) {`
 - `setTimeout(function() {`
 - `console.log(messaggio);`
 - `}, tempo);`
 - `}`
-
- `delayMessage("Hello!", 2000); // Stampa "Hello!" dopo 2 secondi`

Javascript e l'AI

JavaScript è stato aggiornato per renderlo compatibile con le ultime tecnologie, infatti adesso è possibile integrare l'AI:

- **Librerie Edge-Native:** Nuove librerie permettono di far girare modelli di linguaggio (LLM) direttamente nel browser o sui server edge usando WebGPU, riducendo drasticamente i costi di latenza e infrastruttura.
- **Strumenti di Build assistiti:** Tool come **Biome** (che sta sostituendo Prettier e ESLint) usano l'analisi statica avanzata per correggere il codice quasi in tempo reale mentre scrivi.